

## Service Discovery, API Gateway & Circuit Breaker

**How the three fit together.** Service discovery answers *"where is service X right now?"* The gateway answers *"how does the outside world get in, and what cross-cutting rules apply?"* The circuit breaker answers *"what do I do when service X is down so I don't take everything else down with it?"* A request hits the gateway, the gateway uses discovery to find the target, and circuit breakers protect each downstream call.

```
// ---- Eureka Server ----
@SpringBootApplication
```

```

@EnableEurekaServer // turns this app into the registry
public class DiscoveryServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(DiscoveryServerApplication.class, args);
    }
}

# discovery-server application.yml
server:
  port: 8761
eureka:
  client:
    register-with-eureka: false # the server doesn't register with itself
    fetch-registry: false

// ---- A client service (member-service) ----
@SpringBootApplication // @EnableDiscoveryClient is auto-enabled
public class MemberServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(MemberServiceApplication.class, args);
    }
}

// Calling another service BY LOGICAL NAME, not host:port
@Configuration
public class HttpConfig {
    @Bean
    @LoadBalanced // intercepts the URI, resolves via discovery
    public RestClient.Builder restClientBuilder() {
        return RestClient.builder();
    }
}

@Service
public class ClaimsClient {
    private final RestClient client;
    public ClaimsClient(RestClient.Builder builder) {
        this.client = builder.build();
    }
    public ClaimSummary fetch(String memberId) {
        // "claims-service" is the registered logical name - resolved at call time
        return client.get()
            .uri("http://claims-service/api/claims/{id}", memberId)
            .retrieve()
            .body(ClaimSummary.class);
    }
}

# member-service application.yml
spring:
  application:
    name: member-service # this is the logical name in the registry
eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/
  instance:
    prefer-ip-address: true

```

### Real scenario (CareFirst member portal).

The CareFirst portal backend is decomposed into member-service, claims-service, eligibility-service, and benefits-service. During open-enrollment traffic spikes, claims-service autoscales from 2 to 8 instances. Because the portal calls `http://claims-service/...` by logical name and the `@LoadBalanced` client resolves through Eureka, the new instances start receiving traffic within a registry-refresh cycle — no config change, no redeploy of the caller.

### Debugging service discovery:

- **Service not found / UnknownHostException: claims-service** — the caller isn't a discovery client, or the `@LoadBalanced` bean is missing so the logical name never gets resolved. Check that the URI uses the registered `spring.application.name`, not a path.

- **Instance still routed to after it died** — registry staleness. Two lag layers: the server's eviction window and the client's cached copy. Total stale window can exceed a minute with defaults. Lower `eureka.instance.lease-renewal-interval-in-seconds` and the client `registry-fetch-interval-seconds` for faster propagation, at the cost of more chatter.

- **"EMERGENCY! Eureka may be incorrectly claiming instances are up"** — self-preservation mode kicked in. Fine in prod, annoying in dev — disable with `eureka.server.enable-self-preservation: false` in lower environments only.

- Hit `http://localhost:8761` (the Eureka dashboard) and `/eureka/apps` to see exactly what's registered and each instance's status.

### Interview Q&A.

**Q: How does a client know which instance to call if there are five?** Client-side load balancing. The client fetches the full registry and a LoadBalancer (Spring Cloud LoadBalancer, round-robin by default) picks an instance per request. The server isn't a proxy — it just hands out the list; the routing decision is local to the caller, removing a network hop and a single point of failure.

**Q: What happens between an instance dying and traffic stopping?** There's a propagation lag with two parts — the server's lease-expiry eviction and the client's cached-registry refresh. During that window calls can still route to the dead instance and fail, which is exactly why you pair discovery with a circuit breaker and retries.

## 2. API Gateway (Spring Cloud Gateway)

### Definition.

An API gateway is a single entry point that routes incoming requests to backend services and applies cross-cutting concerns — auth, rate limiting, CORS, header rewriting — in one place.

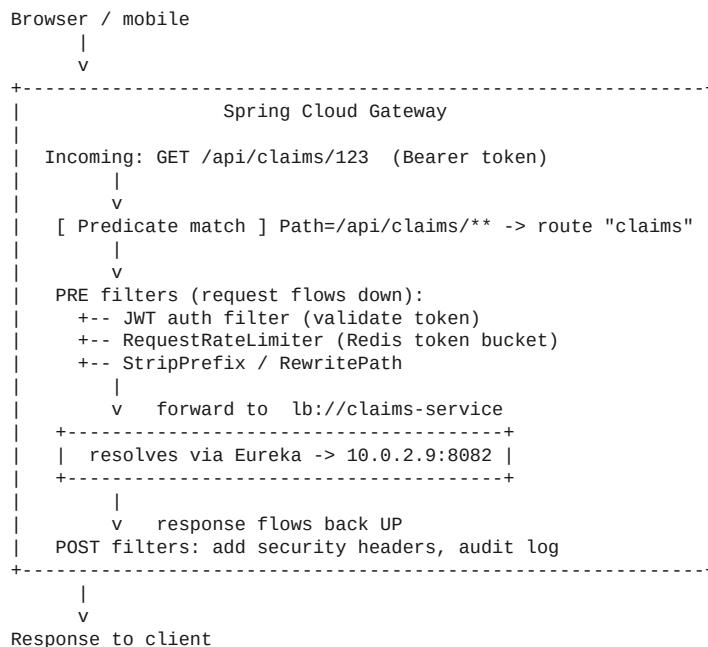
### How it works.

Spring Cloud Gateway matches each request against route definitions using *predicates* (path, host, method, header), then runs the request through a chain of *filters* before forwarding to the target URI and running response filters on the way back. It's built on Project Reactor / Netty, so it's fully non-blocking. Combined with discovery, the target URI can be lb://service-name, resolved through Eureka.

### Why it matters (production impact).

Without a gateway, every service re-implements auth, CORS, and rate limiting — inconsistently. The gateway centralizes the security and traffic perimeter, hides internal topology from clients, and gives you one place to enforce policy. For a regulated healthcare portal, that single enforcement point is also where audit and compliance controls live.

### ASCII diagram - predicate match -> filter chain -> route:



### Working code example (Java / config):

```
# api-gateway application.yml
spring:
  application:
    name: api-gateway
  cloud:
    gateway:
      discovery:
        locator:
          enabled: true          # auto-create routes from registry (optional)
      routes:
        - id: claims-route
          uri: lb://claims-service # lb:// => load-balanced via discovery
          predicates:
            - Path=/api/claims/**
          filters:
            - StripPrefix=1      # /api/claims/123 -> /claims/123 downstream
            - name: RequestRateLimiter
              args:
```

```

        redis-rate-limiter.replenishRate: 50
        redis-rate-limiter.burstCapacity: 100
- id: member-route
  uri: lb://member-service
  predicates:
    - Path=/api/members/**
  filters:
    - StripPrefix=1
server:
  port: 8080

// A custom global filter - runs on EVERY route (JWT validation + correlation ID)
@Component
public class AuthGlobalFilter implements GlobalFilter, Ordered {

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        String token = exchange.getRequest().getHeaders().getFirst(HttpHeaders.AUTHORIZATION);

        if (token == null || !isValid(token)) {
            exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
            return exchange.getResponse().setComplete(); // short-circuit, not forwarded
        }

        // stamp a correlation id so the call is traceable across services
        ServerHttpRequest mutated = exchange.getRequest().mutate()
            .header("X-Correlation-Id", UUID.randomUUID().toString())
            .build();

        return chain.filter(exchange.mutate().request(mutated).build());
    }

    private boolean isValid(String bearer) { /* verify JWT signature + expiry */ return true; }

    @Override
    public int getOrder() { return -1; } // run early in the chain
}

```

### Real scenario (CareFirst member portal).

The CareFirst portal exposes a single origin (/api/\*\*) to the browser. The gateway validates the member's JWT once at the edge and forwards a correlation ID downstream, so member-service -> claims-service calls are traceable end-to-end through Spring's tracing. Rate limiting at the gateway protects eligibility-service (which calls a slow external payer system) from being hammered during enrollment season. The browser never learns the internal topology — it only ever sees the gateway origin, which also keeps CORS configuration in exactly one place.

### Debugging the gateway:

- **404 on a route that should match** — predicate ordering or path mismatch. Routes are evaluated top-down; a broad Path=/\*\* route above a specific one swallows everything. Enable logging.level.org.springframework.cloud.gateway: DEBUG to see which route matched.
- **StripPrefix confusion** — the downstream service receives a different path than you think. StripPrefix=1 removes the first segment; verify the actual forwarded path in debug logs.
- **502 / connection refused to lb://service** — discovery integration. Either the gateway isn't a discovery client, or the target name doesn't match a registered spring.application.name. Check the Eureka dashboard.
- **Rate limiter not limiting** — the RequestRateLimiter needs a KeyResolver bean (what to bucket by) and a reachable Redis; without the resolver it silently no-ops.
- **CORS errors even though configured** — CORS set both at the gateway and a downstream service causes duplicate/conflicting headers. Configure it once, at the gateway.

### Interview Q&A.

**Q: Predicates vs filters?** Predicates decide *whether* a route matches (path, host, method, headers, time). Filters decide *what happens* to the request/response once matched (rewrite path, add headers, rate limit, auth). Predicates are the routing condition; filters are the processing pipeline.

**Q: Why is Spring Cloud Gateway reactive and does that matter?** It's built on Netty/Project Reactor, so it handles requests non-blocking on a small event-loop thread pool rather than one-thread-per-request. For a gateway fronting many slow-ish downstream calls, that means far better throughput under concurrency without exhausting threads — which is precisely the gateway's job profile.

### 3. Circuit Breaker (Resilience4j)

#### Definition.

A circuit breaker is a resilience pattern that stops calling a failing service after a failure threshold, returning a fallback immediately instead of waiting for repeated timeouts.

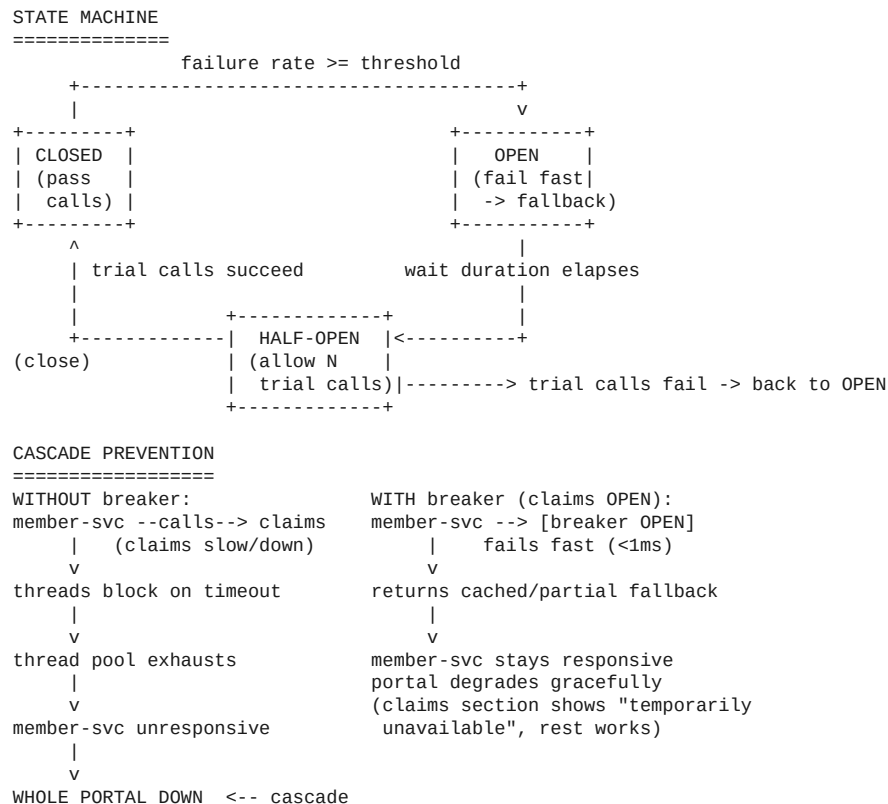
#### How it works.

The breaker wraps a call and tracks success/failure rates in a sliding window. Three states: **CLOSED** (calls pass through, failures counted), **OPEN** (failure rate exceeded the threshold — calls fail fast and hit the fallback without touching the downstream), and **HALF-OPEN** (after a wait period, a few trial calls are allowed; success closes it, failure re-opens it). Resilience4j implements this in a lightweight, functional, non-blocking way and composes with retry, timeout, and bulkhead.

#### Why it matters (production impact).

Without a breaker, a slow downstream service causes callers to pile up blocked threads waiting on timeouts; those threads exhaust, the caller becomes unresponsive, and the failure *cascades* up the chain. The breaker contains the blast radius — one service degrading becomes a graceful fallback instead of a portal-wide outage.

#### ASCII diagram - state machine + cascade prevention:



#### Working code example (Java / Resilience4j):

```
@Service
public class ClaimsClient {

    private final RestClient client;
    public ClaimsClient(RestClient.Builder builder) { this.client = builder.build(); }

    // name "claims" links to config below; fallbackMethod runs when OPEN or on failure
    @CircuitBreaker(name = "claims", fallbackMethod = "claimsFallback")
    @Retry(name = "claims") // retry transient failures before tripping
    public ClaimSummary fetch(String memberId) {
        return client.get()
```

```

        .uri("http://claims-service/api/claims/{id}", memberId)
        .retrieve()
        .body(ClaimSummary.class);
    }

    // Signature must match the original + a Throwable parameter
    private ClaimSummary claimsFallback(String memberId, Throwable t) {
        // graceful degradation: serve last-known/cached or an "unavailable" marker
        return ClaimSummary.unavailable(memberId);
    }
}

# application.yml - Resilience4j config
resilience4j:
  circuitbreaker:
    instances:
      claims:
        sliding-window-type: COUNT_BASED
        sliding-window-size: 10
        failure-rate-threshold: 50          # >=50% of last 10 calls fail -> OPEN
        wait-duration-in-open-state: 10s   # stay OPEN 10s, then HALF-OPEN
        permitted-number-of-calls-in-half-open-state: 3
        slow-call-duration-threshold: 2s    # calls slower than 2s count as "slow"
        slow-call-rate-threshold: 80
  retry:
    instances:
      claims:
        max-attempts: 3
        wait-duration: 200ms
  timelimiter:
    instances:
      claims:
        timeout-duration: 3s

```

### Real scenario (CareFirst member portal).

The portal dashboard aggregates member info, eligibility, and recent claims on one screen. claims-service depends on a legacy mainframe adapter that periodically slows to multi-second responses. Before the breaker, those slow calls blocked the dashboard's threads and the *entire* dashboard would hang — eligibility and member info that were perfectly healthy became unreachable. With a Resilience4j breaker around the claims call, when claims latency crosses the slow-call threshold the breaker opens, the claims panel renders "Claims temporarily unavailable," and the rest of the dashboard loads normally. The breaker auto-probes via HALF-OPEN and restores the panel once the mainframe recovers — no human intervention, no redeploy.

### Debugging the circuit breaker:

- **Fallback never triggers** — the exception type is excluded, @Retry is masking it, or the method is called *within the same class* (Spring AOP proxies don't intercept self-invocation — the call must come through the injected bean). This self-invocation gotcha is the single most common one.
- **Breaker opens too eagerly / too late** — window misconfiguration. A sliding-window-size of 10 with a 50% threshold means 5 failures trip it; that's very sensitive for a high-traffic endpoint. Size the window to your traffic volume.
- **Breaker stuck OPEN** — downstream genuinely still failing on HALF-OPEN trial calls, so it re-opens. Check the trial-call results, not the breaker.
- **Annotations silently ignored** — missing spring-cloud-starter-circuitbreaker-resilience4j (or the AOP starter), so the aspect isn't applied.
- **Observability** — expose /actuator/circuitbreakers and the /actuator/health circuit-breaker indicator, and watch state-transition metrics in Micrometer/Prometheus. Alert on OPEN transitions.

### Interview Q&A.

**Q: Walk me through the three states.** CLOSED: calls flow, failures tracked in a sliding window. When the failure (or slow-call) rate crosses the threshold, it trips to OPEN: calls fail fast into the fallback without



touching the downstream, for a configured wait duration. Then HALF-OPEN: a limited number of trial calls are allowed; success closes the breaker, failure sends it back to OPEN. The HALF-OPEN probe is what makes recovery automatic.

**Q: Circuit breaker vs retry - aren't they opposed?** Complementary at different layers. Retry handles *transient* blips (a dropped packet, a brief GC pause) by trying again quickly. The breaker handles *sustained* failure by stopping retries once a service is clearly down — otherwise retries amplify load on a struggling service. Order matters: retry sits inside the breaker, so exhausted retries count as one failure toward tripping it, and once OPEN you stop retrying altogether.

**Q: Where does the fallback data come from?** Whatever degrades gracefully — a cache of last-known-good data, a partial response, a sensible default, or an explicit "unavailable" marker the UI handles. The fallback must not itself call the failing service.

## 4. How the three compose

A browser request hits the **gateway**, which authenticates it once and matches it to a route targeting `lb://claims-service`. The `lb://` scheme resolves through **service discovery** to a live, load-balanced instance. The downstream `member-service -> claims-service` call is wrapped in a **circuit breaker** so that if `claims` is unhealthy, the dashboard degrades gracefully instead of cascading into a full outage. Discovery handles *where*, the gateway handles *entry and policy*, the breaker handles *failure*. Each compensates for a weakness the others can't: discovery's propagation lag is covered by the breaker; the gateway's single-entry risk is covered by running multiple gateway instances (themselves discovery-registered).

```
Browser
|
v
[ API Gateway ] --auth, rate limit, correlation id-->
| route: lb://claims-service
v
[ Service Discovery ] --resolves lb:// to a live instance-->
|
v
[ member-service ] --(circuit breaker)--> [ claims-service ]
|
v (if OPEN)
fallback: cached / "unavailable"
```

## 5. Version / dependency caveat

The specific defaults and starter coordinates shift between Spring Cloud release trains, and Netflix Eureka/Ribbon's status has changed over the years (Ribbon is retired; Spring Cloud LoadBalancer replaced it). If you're standing this up fresh, check the current Spring Cloud release-train docs for the exact dependency versions and any deprecations before committing config.